Solution: Move Zeroes

Problem Statement

Given an array of integers `nums`, move all the `0`s present in the array to the end while maintaining the relative order of the non-zero elements.

Note: This rearrangement should be done in-place without using extra space for another array.

Example 1:

- **Input**: `[1, 0, 2, 0, 3, 0, 4]`
- **Expected Output**: `[1, 2, 3, 4, 0, 0, 0]`
- **Justification**: Here, all non-zero elements (1, 2, 3, 4) retain their order, and all zeros are moved to the end of the array.

Example 2:

- **Input**: `[0, 0, 0, 10, 20]`
- **Expected Output**: `[10, 20, 0, 0, 0]`
- **Justification**: The non-zero elements (10, 20) are shifted to the front, and the zeros are relocated to the end.

Example 3:

- **Input**: `[5, 1, 0, 2, 0]`
- **Expected Output**: `[5, 1, 2, 0, 0]`
- **Justification**: Non-zero elements (5, 1, 2) maintain their sequence, while zeros are moved to the end.

Solution

To solve this problem, we will use a two-pointer approach. The first pointer (`i`) will iterate over the array, and the second pointer (`lastNonZeroIndex`) will keep track of the position where the next non-zero element should be placed.

This approach is efficient as it allows us to perform the operation in-place, reducing the space complexity. It's also effective because it ensures that the relative order of non-zero elements is maintained by sequentially filling non-zero elements from the start. As we iterate through the array, we swap non-zero elements with the element at the `lastNonZeroIndex` and increment `lastNonZeroIndex`. This method is optimal as it requires a single pass over the array, ensuring linear time complexity.

Step-by-step algorithm

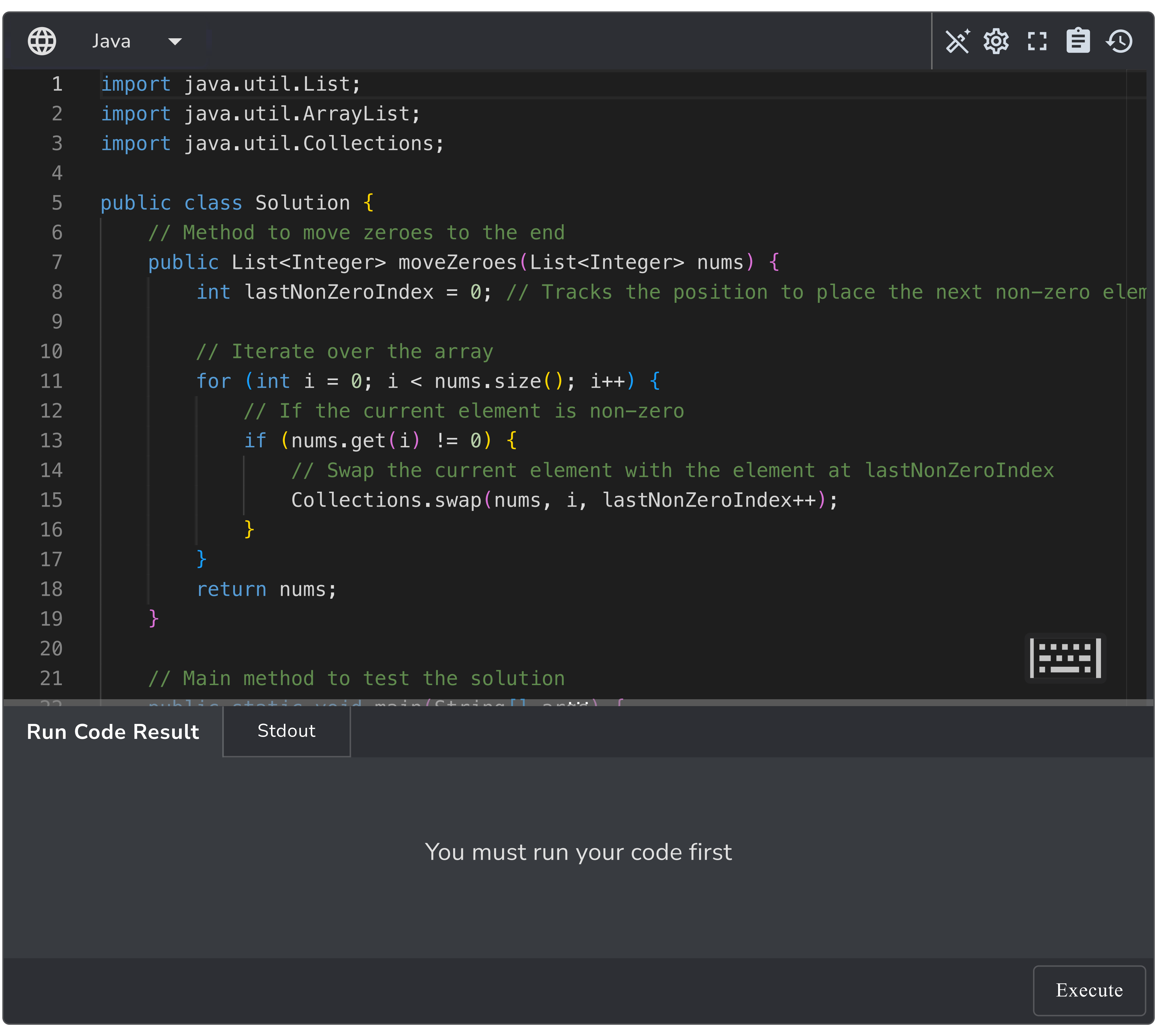
- Initialize `lastNonZeroIndex` to 0.
- Iterate through each element (`i`) in the array:
  - If the current element is not zero:
    - Swap the element at `i` with the element at `lastNonZeroIndex`.
    - Increment `lastNonZeroIndex`.
- This process groups all non-zero elements at the beginning of the array and zeros at the end, maintaining their original order.

Algorithm Walkthrough

Let's take Example 1 `[1, 0, 2, 0, 3, 0, 4]`:

- Start with `lastNonZeroIndex = 0` and `i = 0`.
- Iterate through the array:
  - At `i = 0`, element 1 is non-zero, swap it with itself, and increment `lastNonZeroIndex` to 1.
  - At `i = 1`, element 0 is zero, continue.
  - At `i = 2`, element 2 is non-zero, swap it with the first 0 (at index `lastNonZeroIndex`, which is 1), `lastNonZeroIndex` becomes 2.
  - At `i = 3`, element 0 is zero, continue.
  - At `i = 4`, element 3 is non-zero, swap it with the 0 (at index `lastNonZeroIndex`, which is 2), `lastNonZeroIndex` becomes 3.
  - At `i = 5`, element 0 is zero, continue.
  - At `i = 6`, element 4 is non-zero, swap it with the 0 (at index `lastNonZeroIndex`, which is 3), `lastNonZeroIndex` becomes 4.
- Resulting array after completion: `[1, 2, 3, 4, 0, 0, 0]`.

Code



```java
import java.util.List;
import java.util.ArrayList;
import java.util.Collections;

public class Solution {
    // Method to move zeroes to the end
    public List<Integer> moveZeroes(List<Integer> nums) {
        int lastNonZeroIndex = 0; // Tracks the position to place the next non-zero elem

        // Iterate over the array
        for (int i = 0; i < nums.size(); i++) {
            // If the current element is non-zero
            if (nums.get(i) != 0) {
                // Swap the current element with the element at lastNonZeroIndex
                Collections.swap(nums, i, lastNonZeroIndex++);
            }
        }
        return nums;
    }

    // Main method to test the solution
```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(n), where n is the length of the array, as We iterate through the array once.
- **Space Complexity**: O(1), as the rearrangement is done in-place without using extra space.

← Previous
Move Zeroes (Easy)
Next →
Missing Ranges (Easy)
☐ Mark as Completed